

CQt day2 hw2 레포트

로봇 예비단원 주호진

1. 과제

- Queue 또는 stack 기능을 활용한 UI 제작

Queue : FIFO(First In First Out), 제일 처음 넣은 데이터가 처음으로 빠져나오는 구조

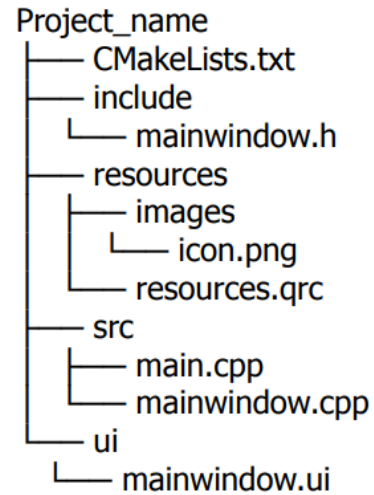
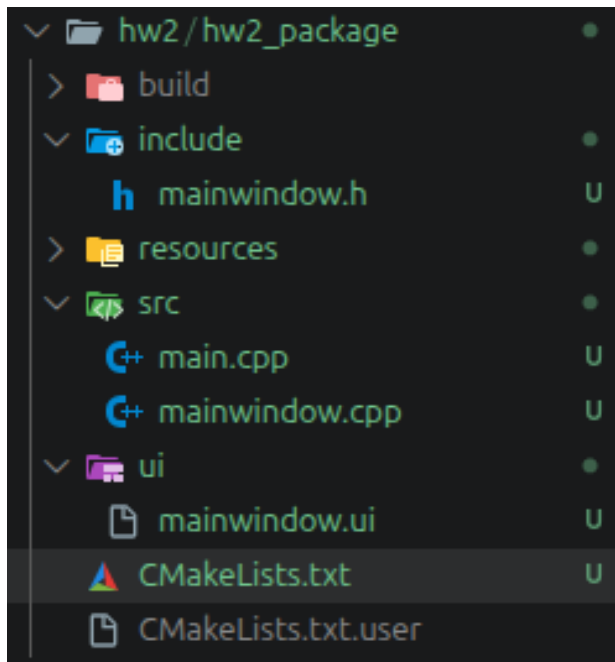
Stack : LIFO(Last In First Out), 제일 마지막에 넣은 데이터가 처음으로 빠져나오는 구조

- C++ 개념 중 벡터를 활용하여 구현할 것

- UI는 자유주제로 제작(ex - redo/undo기능 활용한 그림판 등)

2. 폴더 구조

```
++
hojin@hojins:~$ cd "/home/hojin/robit/21th_robit_intern_Hojin_Joo/C++Qt/day2/hw2/hw2_package"
hojin@hojins:~/robit/21th_robit_intern_Hojin_Joo/C++Qt/day2/hw2/hw2_package$ mkdir -p include resources/images src ui
touch resources/resources.qrc resources/images/test.png
hojin@hojins:~/robit/21th_robit_intern_Hojin_Joo/C++Qt/day2/hw2/hw2_package$ mv mainwindow.h include/
mv main.cpp mainwindow.cpp src/
mv mainwindow.ui ui/
```



위와 같은 구조로 설정

폴더 구조는 ppt에 제시된 구조대로 설정하였고 .gitignore을 통해 build 폴더와 CMakeLists.txt.user 파일은 변경 사항에 적용되지 않도록 했다.

3. 코드 설명

3-1. Mainwindow.h

```

#include <vector>

```

Stack 구현을 위한 vector를 include했다.

```

class ScribbleArea : public QWidget
{
    Q_OBJECT

public:
    ScribbleArea(QWidget *parent = nullptr);
    void undo();
    void redo();

protected:
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
    void paintEvent(QPaintEvent *event) override;
    void resizeEvent(QResizeEvent *event) override;

private:
    void drawLineTo(const QPoint &endPoint);
    void resizeImage(QImage *image, const QSize &newSize);

    bool scribbling = false;
    int myPenWidth = 1;
    QColor myPenColor = Qt::blue;
    QImage image;
    QPoint lastPoint;

    std::vector<QImage> undoStack;
    std::vector<QImage> redoStack;
    QImage cacheImage;
};

```

<https://doc.qt.io/qt-6/qtwidgets-widgets-scribble-example.html>

Scribble이라는 이미지 위에 그림을 그리는 예제가 공식 문서 예제에 있는데 이미지를 불러오거나 저장하는 기능은 해당 과제에서 필요 없기 때문에 해당 선언은 제외하고 가져왔다.

```

void undo();    std::vector<QImage> undoStack;
void redo();    std::vector<QImage> redoStack;
               QImage cacheImage;

```

해당 코드는 예제 외에 stack을 구현하기 위해 추가된 코드이다.

3-2. Mainwindow.cpp

```

QWidget *Widget = new QWidget(this);

QHBoxLayout *mainLayout = new QHBoxLayout(Widget); // H: horizon

QVBoxLayout *buttonLayout = new QVBoxLayout(); // V: vertical

QPushButton *undoButton = new QPushButton("Undo", this);
QPushButton *redoButton = new QPushButton("Redo", this);

buttonLayout->addWidget(undoButton);
buttonLayout->addWidget(redoButton);
buttonLayout->addStretch(); // 빈 공간을 채움
// buttonLayout에 vertical로 undoBtn, redoBtn을 배치(stretch 포함)

ScribbleArea *scribbleArea = new ScribbleArea(this);

mainLayout->addLayout(buttonLayout);
mainLayout->addWidget(scribbleArea, 1);
// mainLayout에 horizon으로 버튼과 scribble 화면을 배치

setCentralWidget(Widget);

// button의 clicked signal을 각각 undo, redo 함수와 연결)
connect(undoButton, &QPushButton::clicked, scribbleArea, &ScribbleArea::undo);
connect(redoButton, &QPushButton::clicked, scribbleArea, &ScribbleArea::redo);

```

main화면에 scribblearea와 버튼을 배치하기 위한 코드, 컨테이너 개념의 박스를 만들어서 vertical과 horizon을 배치함.

```

void ScribbleArea::mousePressEvent(QMouseEvent *event)
{
    // 좌클릭 event가 감지되면(눌림)
    if (event->button() == Qt::LeftButton) {
        cacheImage = image;
        lastPoint = event->pos();
        scribbling = true;
    }
}

void ScribbleArea::mouseMoveEvent(QMouseEvent *event)
{
    // 그리는 중
    if ((event->buttons() & Qt::LeftButton) && scribbling)
        drawLineTo(event->pos());
}

void ScribbleArea::mouseReleaseEvent(QMouseEvent *event)
{
    // 좌클릭 event가 감지되면(떼짐)
    if (event->button() == Qt::LeftButton && scribbling) {
        drawLineTo(event->pos());
        scribbling = false;

        undoStack.push_back(cacheImage);
        redoStack.clear();
    }
}

```

마우스 좌클릭 event를 감지하여 그리기를 구현하는 부분, [예제](#)를 참고하였다.

```

void ScribbleArea::undo()
{
    if (undoStack.empty()) // 비어있으면 무시
        return;

    redoStack.push_back(image); // redo stack에 push(redo용)
    image = undoStack.back(); // 이전 image(back)를 image에 저장
    undoStack.pop_back(); // pop

    update();
}

void ScribbleArea::redo()
{
    if (redoStack.empty())
        return;

    undoStack.push_back(image);
    image = redoStack.back();
    redoStack.pop_back();

    update();
}

```

Undo, redo 함수 부분. Undo를 호출했을 때 stack을 먼저 확인한 뒤 스택이 있다면 redo stack에 현재 정보를 push한 뒤 이전 정보를 불러오고 기존 정보는 pop한다. (redo도 같은 로직)

4. 실행 결과 화면

[실행 영상](#)

5. AI 사용 보고서

<https://share.gemini.google/DccD5Btcbj90>

답변이 긴 질문은 질문 캡처본만 올려서 정리하였다.

<https://doc.qt.io/qt-6/qtwidgets-widgets-scribble-example.html>

이걸로 지금 획 그리는 거까지 구현했는데 획을 stack으로 관리하려면 어떻게 해야 돼

vector 사용해서 구현

예제로 그리기 화면을 구현한 뒤에 stack을 적용하는 부분을 질문했다. 일단 qt 파일 구조에서 main.cpp, mainwindow.cpp가 나뉘어져 있기 때문에 어디에 구현해야하는지부터 헷갈렸다. 또한 vector를 사용해서 구현해보는 것이 처음이었기에 vector 예제도 물어보았다.

Undo redo를 구현하기 위해서는 각각마다 따로 stack을 선언해서 관리해주어야 하는 구조임을 파악하였다.

ScribbleArea이 mainwindow central widget인데 다른 버튼과 horizon으로 배치하려면 .ui에서 어떻게 해

QMainWindow는 단 하나의 중앙 위젯(Central Widget)만 가질 수 있으므로, .ui 파일에서 수평 레이아웃(QHBoxLayout)이 적용된 컨테이너 위젯을 활용해 버튼과 ScribbleArea를 나란히 배치해야 합니다.

undo와 redo를 구현하기 위해서 scribble(그림 그리는) 화면 외에 버튼을 배치해야 하는데 기존에 central widget으로 scribble area를 설정해놓은 상태에서 .ui파일 상에 버튼을 추가해도 보이지 않아 질문하게 되었다. Central widget은 하나만 선언할 수 있기 때문에, Mainwindow.cpp 상에서 버튼을 추가해서 widget을 원하는 모양으로 배치한 뒤에 해당 widget을 central widget으로 설정하는 방식을 사용해야 하는 것을 알았다.